

UNIDAD 3

PROGRAMACIÓN ESTRUCTURADA CON JAVA

**Programación
CFGS DAW**

Adaptado por: José Miguel Blázquez
2022/2023

Licencia



Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original. NOTA: Esta es una obra adaptada por José Miguel Blázquez, derivada de la obra original realizada por Carlos Cacho, Raquel Torres, Lionel Tarazón y Fco. Javier Valero.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE DE CONTENIDOS

1. El control de flujo.....	4
2. Estructuras condicionales.....	4
2.1 Estructura condicional Simple (if).....	5
2.2 Estructura condicional Doble (if-else).....	6
2.3 Estructura condicional múltiple (switch).....	8
2.4 El operador ternario "?:" en Java.....	10
2.5 Ejemplo de condicional.....	11
3. Estructuras repetitivas o iterativas.....	12
3.1 Bucle WHILE.....	13
3.2 Bucle DO-WHILE.....	15
3.3 Bucle FOR.....	17
3.4 Comparativa de bucles.....	19
4. Ejemplos de bucles.....	20
4.1 Ejemplo 1.....	20
4.2 Ejemplo 2.....	21
5. Operaciones avanzadas con estructuras de control.....	23
5.1 Aritmética de control en bucles.....	23
5.2 Contadores y <i>flags</i> (variables bandera).....	23
5.3 Bucles infinitos.....	25
5.4 Condicionales encadenados y condicionales anidados.....	26
5.5 Bucles anidados.....	27
5.6 Sentencias "break" y "continue".....	27
5.7 Funcionamiento especial de switch.....	29
6. Agradecimientos.....	29

UD3. PROGRAMACIÓN ESTRUCTURADA CON JAVA

1. EL CONTROL DE FLUJO

Los programas se ejecutan siguiendo un flujo ordenado secuencial. Sin embargo, existen muchas situaciones en que necesitamos programar alteraciones de dicho flujo según condiciones, bien eligiendo qué partes del código se ejecutarán, o bien ejecutando de forma reiterada ciertas partes de código. Para ello, la programación estructurada nos proporciona las siguientes estructuras:

- Condicionales (o alternativas)
 - Condicional simple (IF)
 - Condicional doble (IF-ELSE)
 - Condicional múltiple (SWITCH)
- Repetitivas (también llamadas iterativas o bucles)
 - Bucle WHILE
 - Bucle DO-WHILE
 - Bucle FOR

2. ESTRUCTURAS CONDICIONALES

Existen estructuras que permiten alterar el flujo secuencial de un programa de forma que en función de una condición o el valor de una expresión, el mismo pueda ser desviado en una u otra alternativa de código. Veamos en primer lugar las alternativas, condicionales o “de selección”.

Las estructuras condicionales disponibles en Java son:

- Condicional Simple (if)
- Condicional Doble (if-else)
- Condicional Múltiple (switch)

2.1 Estructura condicional Simple (if)

La condicional simple se codifica de la siguiente forma:

Código	Ordinograma
<pre>if (condición) sentencia; if (condición){ //bloque }</pre> La sentencia o el bloque de Acciones se ejecuta si la condición se evalúa a true (verdadera). <pre>if (cont == 0) { System.out.println("cont es 0"); // más instrucciones... }</pre> Si dentro del if solo hay una instrucción, no es necesario poner las llaves. <pre>if (cont == 0) System.out.println("cont es 0");</pre>	<pre>graph TD Entry(()) --> Condicion{Condición} Condicion -- cierta --> Bloque[Bloque] Condicion -- falsa --> Join(()) Bloque --> Join Join --> Exit(())</pre>

2.2 Estructura condicional Doble (if-else)

Código

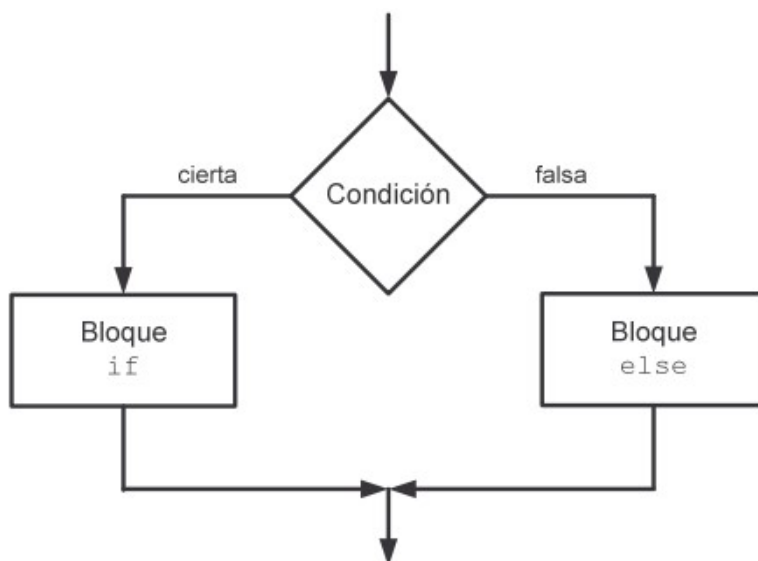
```
if (condición){  
    // Acciones verdadero;  
}  
else{  
    // Acciones falso;  
}
```

El bloque Acciones verdadero se ejecuta si la condición se evalúa a true (verdadera). En caso contrario, se ejecuta el bloque de Acciones falso.

```
if (cont == 0){  
    System.out.println("cont es 0");  
    // más instrucciones...  
}  
else{  
    System.out.println("cont no es 0");  
    // más instrucciones...  
}
```

Si sólo hay 1 instrucción por bloque, no es necesario poner las llaves.

```
if (cont == 0)  
    System.out.println("cont es 0");  
else  
    System.out.println("cont no es 0");
```



⚡ Recordad que el operador relacional para comprobar si son iguales es `==`, no un solo `=` que corresponde con el operador de asignación. Este error no lo detecta el compilador y es difícil de averiguar.

En muchas ocasiones, se **anidan** estructuras condicional if-else, de forma que se pregunte por una condición si anteriormente no se ha cumplido otra y así sucesivamente. Por ejemplo: supongamos que realizamos un programa que muestra la nota de un alumno en la forma (insuficiente, suficiente, bien, notable o sobresaliente) en función de su nota numérica.

Podría codificarse de la siguiente forma:

```
1
2 import java.util.Scanner;
3
4 public class Nota {
5
6     public static void main(String[] args) {
7         Scanner entrada = new Scanner(System.in);
8         int nota;
9         //Suponemos que el usuario introduce el número correctamente.
10        //No hacemos comprobación
11        System.out.println("Dame un número entre 0 y 10");
12
13        nota = entrada.nextInt();
14
15        if (nota < 5) {
16            System.out.println("Insuficiente");
17        } else if (nota < 6) {
18            System.out.println("Suficiente");
19        } else if (nota < 7) {
20            System.out.println("Bien");
21        } else if (nota < 9) {
22            System.out.println("Notable");
23        } else {
24            System.out.println("Sobresaliente");
25        }
26    }
27 }
28
29
30
31
32
33
34
35
36
37
38 }
```

Siendo la salida:

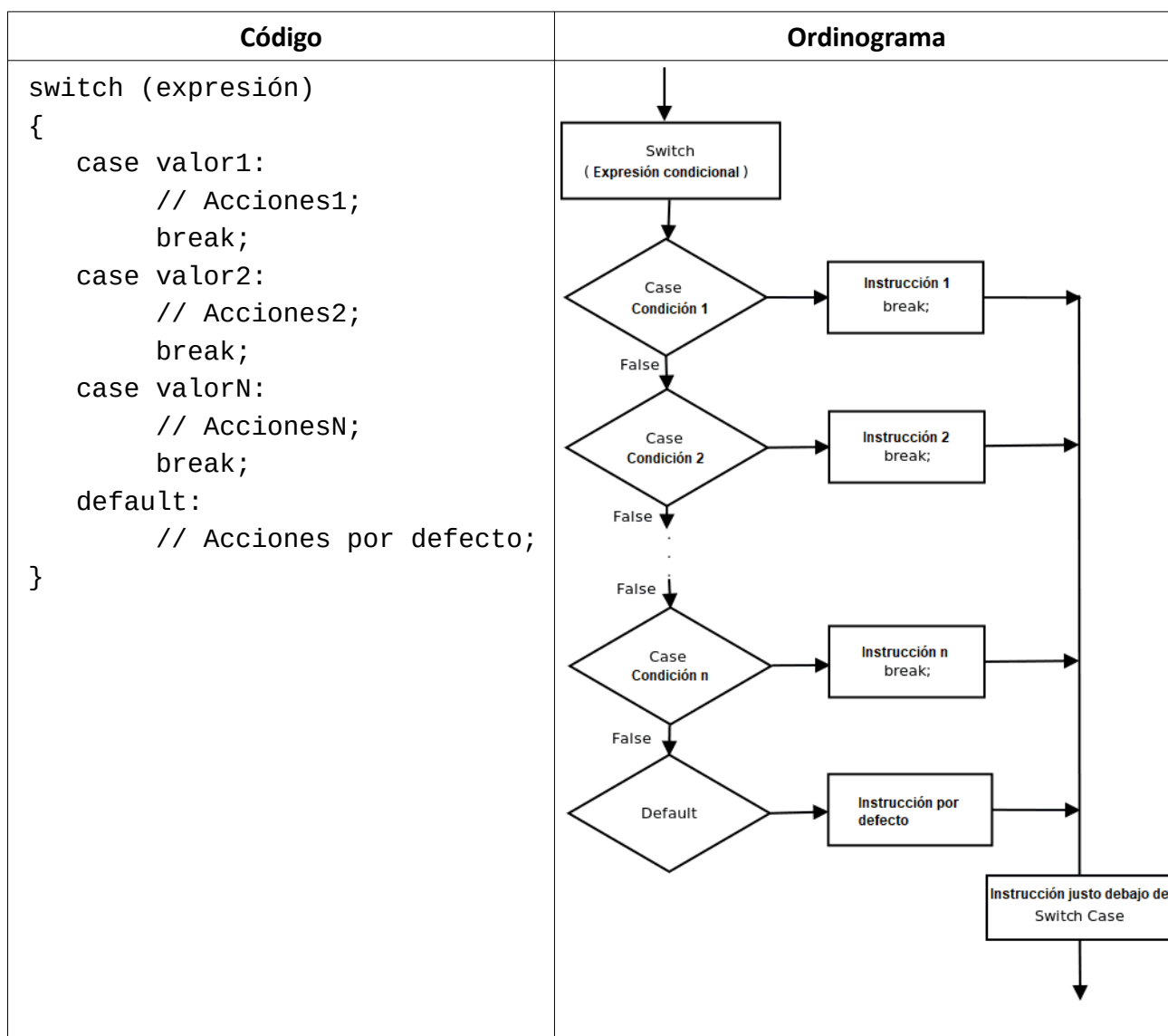
```
run:
Dame un número entre 0 y 10
8
Notable
BUILD SUCCESSFUL (total time: 11 seconds)
```

🔊 Es muy recomendable usar la tecla tabulador en las instrucciones de cada bloque. Como se puede ver en el ejemplo, cada **else** está alineado con su **if** asociado, de esta forma es más fácil leer el código.

Ejercicio de prueba: toma el código del programa para resolver ecuaciones de 2º grado y, empleando condicionales, controla los casos en que no haya solución y en que sólo haya una.

2.3 Estructura condicional múltiple (switch)

La alternativa múltiple se codifica de la siguiente forma:



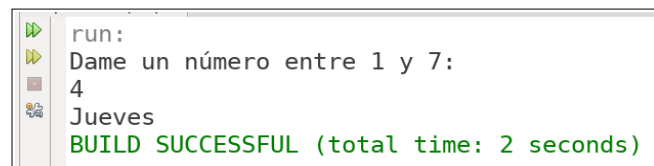
Es muy importante entender que en el switch se evalúa una expresión (un valor concreto como 0, 5, 1...) no una condición (verdadera o falsa) como en el if y el ifelse.

El programa comprueba el valor de la expresión y saltará al 'case' que corresponda con dicho valor (valor1 o valor2 o ...) ejecutando el código de dicho 'case' (Acciones1 o Acciones2 o ...). Si no coincide ningún valor, saltará al 'default' y ejecutará las acciones por defecto.

Es importante añadir la sentencia **break;** al final de cada 'case', ya que de lo contrario el programa seguirá ejecutando el código de las demás acciones y normalmente no queremos que haga eso (aunque Java permite hacerlo, es confuso y por ello está desaconsejado). Un ejemplo sería el siguiente:

```
1
2 import java.util.Scanner;
3
4 public class Alternativa_Multiple {
5
6     public static void main(String[] args) {
7         Scanner entrada = new Scanner(System.in);
8         int dia;
9
10        System.out.println("Dame un número entre 1 y 7:");
11
12        dia = entrada.nextInt();
13
14        switch (dia) {
15            case 1:
16                System.out.println("Lunes");
17                break;
18            case 2:
19                System.out.println("Martes");
20                break;
21            case 3:
22                System.out.println("Miércoles");
23                break;
24            case 4:
25                System.out.println("Jueves");
26                break;
27            case 5:
28                System.out.println("Viernes");
29                break;
30            case 6:
31                System.out.println("Sábado");
32                break;
33            case 7:
34                System.out.println("Domingo");
35                break;
36            default:
37                System.out.println("Error el número debe estar entre 0 y 7");
38        }
39    }
40 }
41
42 }
```

Y la salida:



```
run:
Dame un número entre 1 y 7:
4
Jueves
BUILD SUCCESSFUL (total time: 2 seconds)
```

2.4 El operador ternario “?:” en Java

En ocasiones, un `if` se utiliza simplemente para obtener un valor u otro en función de si una se cumple una condición o no, en estos casos se puede sustituir la estructura `if-else` por el operador **ternario u operador condicional** que tiene la sintaxis `resultado = condicion ? valor_si_se_cumple : valor_si_no_se_cumple` los valores tanto para el caso de que se cumpla como para el que no pueden ser valores directamente como un texto o un entero o llamadas a funciones que devuelvan un objeto del tipo esperado.

```
// Ejemplo 1
String queHacer = (temp > 25) ? "A la playa!!!" : "Esperando buen tiempo...";

// Ejemplo 2
double numero = (temp > 10) ? (Math.random() * 10) : 0;

// Ejemplo 3
Comida miComida = cansado ? pedirComida("china") : hacerComida();
```

Veamos un ejemplo de código real para verificar que una persona es mayor de edad, empleando el operador ternario:

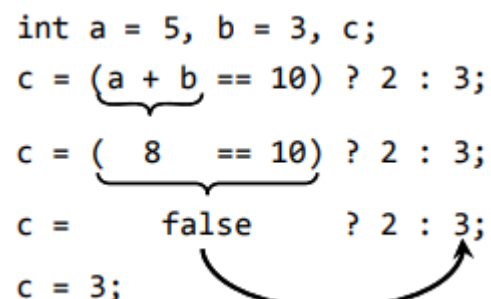
```
boolean mayor_de_edad = edad >= 18 ? true : false;
```

Esa instrucción sería equivalente a todo el bloque que se muestra a continuación, podemos observar el claro ahorro de código:

```
boolean mayor_de_edad;

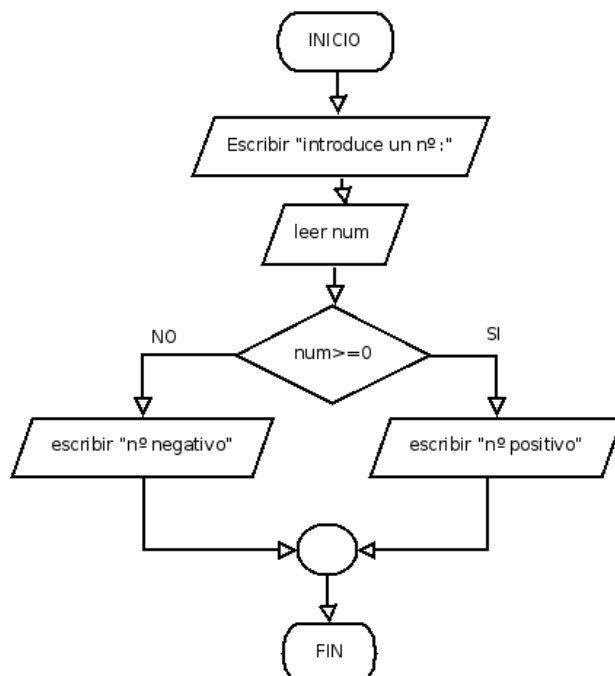
if(edad >= 18){
    mayor_de_edad = true;
}else{
    mayor_de_edad = false;
}
```

```
int a = 5, b = 3, c;
c = (a + b == 10) ? 2 : 3;
c = ( 8 == 10) ? 2 : 3;
c = false ? 2 : 3;
c = 3;
```



2.5 Ejemplo de condicional

Programa que lee un número y dice si es positivo o negativo. Consideramos cero como positivo.



```
1 package ejemplo2;
2
3 import java.util.Scanner; // Importamos la clase Scanner
4
5 public class Ejemplo2 {
6
7     public static void main(String[] args) {
8
9         // Declaramos la variable num
10        int num;
11
12        // Creamos el objeto Scanner para leer por teclado
13        Scanner reader = new Scanner(System.in);
14
15        // Pedimos y leemos x
16        System.out.print("Introduce un nº: ");
17        num = reader.nextInt();
18
19        // Astructura alternativa doble
20        if (num >= 0)
21            System.out.println("Número positivo");
22        else
23            System.out.println("Número negativo");
24    }
25 }
```

```
run:
Introduce un nº: 10
Número positivo
```

3. ESTRUCTURAS REPETITIVAS O ITERATIVAS

Los bucles son estructuras de repetición, bloques de instrucciones que se repiten un número de veces mientras se cumpla una condición o hasta que se cumpla una condición.

Un bloque de instrucciones se encontrará encerrado mediante llaves {...} si existe más de una instrucción al igual que suceden las estructuras alternativas (if... else... etc).

Existen tres construcciones para estas estructuras de repetición:

- Bucle *WHILE*
- Bucle *DO-WHILE*
- Bucle *FOR*

Todo problema que requiera repetición puede hacerse con cualquiera de los tres, pero según el caso suele ser más sencillo o intuitivo utilizar uno u otro.

Como regla general es recomendable:



Utilizar el bucle **for** cuando se conozca de antemano el número exacto de veces que ha de repetirse el bloque de instrucciones.



Utilizar el bucle **while** cuando no sabemos el número de veces que ha de repetirse el bloque y es posible que no deba ejecutarse ninguna vez.



Utilizar el bucle **do-while** cuando no sabemos el número de veces que ha de repetirse el bloque y deberá ejecutarse al menos una vez.

Estas reglas son generales y algunos programadores se sienten más cómodos utilizando principalmente una de ellas. Con mayor o menor esfuerzo, puede utilizarse cualquiera de las tres indistintamente.

3.1 Bucle WHILE

El bucle *while* se codifica de la siguiente forma:

Código	Ordinograma
<pre>while (condición) { bloque acciones; }</pre> La condición debe ser una expresión lógica-relacional que se evalúe a true o false.	<pre>graph TD Entry(()) --> Condicion{Condicion} Condicion -- Verdadero --> Accion1[Accion 1] Accion1 --> Accion2[Accion 2] Accion2 --> Accion3[Accion 3] Accion3 --> AccionN[Accion N] AccionN --> Condicion Condicion -- Falso --> Exit(())</pre>

El bloque de instrucciones se ejecuta mientras se cumple una condición (mientras **condición** se evalúe a true). **La condición se comprueba ANTES de empezar** a ejecutar por primera vez el bucle, por lo que si se evalúa a false en la primera iteración, entonces el bloque de acciones no se ejecutará ninguna vez.

Veamos un ejemplo:

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    int max, cont;  
    System.out.print("Introduce el número máximo: ");  
    max = sc.nextInt();  
    cont = 1;  
    while (cont <= max) {  
        System.out.println("Número: " + cont);  
        cont++;  
    }  
}
```

La salida:

```
run:  
Introduce el número máximo: 5  
Número: 1  
Número: 2  
Número: 3  
Número: 4  
Número: 5  
BUILD SUCCESSFUL (total time: 6 seconds)
```

3.2 Bucle DO-WHILE

El bucle *do-while* se codifica de la siguiente forma:

Código	Ordinograma
<pre>do { bloque acciones; } while (condición);</pre> De la misma forma que el bucle WHILE, la condición debe ser una expresión que se evalúe a true o false.	<pre>graph TD Entry(()) --> A1[Accion 1] A1 --> A2[Accion 2] A2 --> A3[Accion 3] A3 --> AN[Accion N] AN --> Cond{Condicion} Cond -- Verdadero --> Entry Cond -- Falso --> Exit(())</pre>

En este tipo de bucle, **el bloque de instrucciones se ejecuta siempre al menos una vez**, y ese bloque de instrucciones se ejecutará mientras **condición** se evalúe a *true*.

⚡ Por ello en el bloque de instrucciones deberá existir alguna que, en algún momento, haga que *condición* se evalúe a *false*. ¡Si no el bucle no acabaría nunca!

El mismo anterior hecho con un bucle **do-while** sería:

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int max, cont;
    System.out.print("Introduce el número máximo: ");
    max = sc.nextInt();
    cont = 1;

    do {
        System.out.println("Número: " + cont);
        cont++;
    } while (cont <= max);
}
```

Y la salida:

```
run:
Introduce el número máximo: 5
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
BUILD SUCCESSFUL (total time: 6 seconds)
```

3.3 Bucle FOR

El bucle *for* se codifica de la siguiente forma:

Código	Ordinograma
<pre>for (inicialización ; condición ; incremento) { bloque acciones; }</pre>	<pre> graph TD Start(()) --> Init[Iniciar contador] Init --> Cond{Condicion} Cond -- Verdadero --> Acc[Acciones] Acc --> Inc[Incrementar contador] Inc --> Cond Cond -- Falso --> Exit(()) </pre>

La cláusula ***inicialización*** es una instrucción que se ejecuta una sola vez al inicio del bucle, normalmente para inicializar un contador. Por ejemplo ***int i = 1;***

La cláusula ***condición*** es una expresión lógica que se evalúa al inicio de cada iteración del bucle. En el momento en que dicha expresión se evalúe a false se dejará de ejecutar el bucle y el control del programa pasará a la siguiente instrucción (a continuación del bucle *for*). Se utiliza para indicar la condición en la que quieres que el bucle continúe. Por ejemplo ***i <= 10;***

La cláusula ***incremento*** es una instrucción que se ejecuta al final de cada iteración del bucle (después del bloque de instrucciones). Generalmente se utiliza para incrementar o decrementar el contador. Por ejemplo ***i++;*** (incrementar i en 1).

Ejemplo 1: Bucle que muestra por pantalla los números naturales del 1 al 10:

```
for (int i = 1; i <= 10 ; i++) {  
    System.out.println(i);  
}
```

En la inicialización utilizamos **int i=1** para crear la variable *i* con un valor inicial de 1.

La condición **i<=10** indica que el bucle debe repetirse mientras *i* sea menor o igual a 0.

La actualización **i++** indica que, al final de cada iteración, *i* debe incrementarse en 1.

Ejemplo 2: Programa que muestra los números naturales (1,2,3,4,5,6,...) hasta un número introducido por teclado.

```
6 public static void main(String[] args) {  
7     Scanner sc = new Scanner(System.in);  
8     int max;  
9     System.out.print("Introduce el número máximo: ");  
10    max = sc.nextInt();  
11    for (int i = 1; i <= max; i++) {  
12        System.out.println("Número: " + i);  
13    }  
14 }  
15 }  
16 }
```

Siendo la salida:

```
run:  
Introduce el número máximo: 5  
Número: 1  
Número: 2  
Número: 3  
Número: 4  
Número: 5  
BUILD SUCCESSFUL (total time: 6 seconds)
```

Visibilidad de la variable “i”: ámbito local

Los bucles (y los bloques de código que se usan en las estructuras condicionales) generan un ámbito local y privado para la visibilidad de variables. A nivel práctico eso significa que, en el bucle del ejemplo, la variable “*i*” sólo es visible (y existen) dentro del bucle. Si se intenta acceder a ella tras el bucle no podremos, no existe y tampoco es visible aunque existiera.

3.4 Comparativa de bucles

Los siguientes códigos fuente son equivalentes, obsérvalos bien...

WHILE

```
1 | int x = 0;
2 |
3 | while (x < 5) {
4 |     System.out.println(x);
5 |     x = x + 1;
6 | }
```

DO-WHILE

```
1 | int x = 0;
2 | do {
3 |     System.out.println(x);
4 |     x = x + 1;
5 | }
6 | while (x < 5);
```

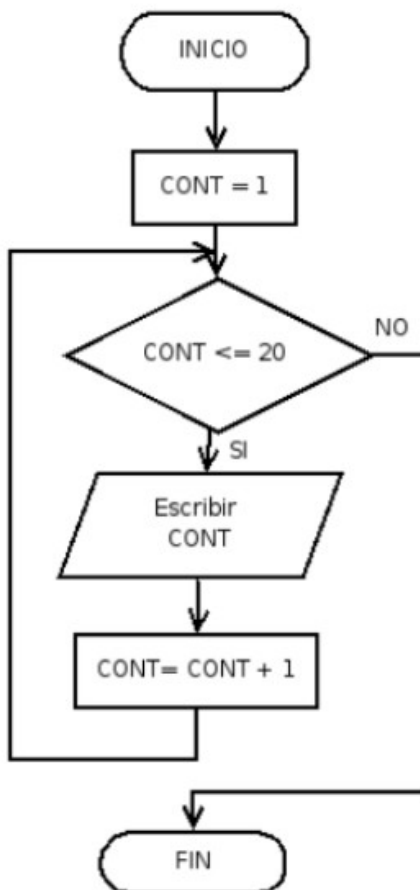
FOR

```
for (int i = 0; i < 5; i++){
    System.out.println(i);
}
```

4. EJEMPLOS DE BUCLES

4.1 Ejemplo 1

Programa que muestre por pantalla los 20 primeros números naturales (1, 2, 3... 20).

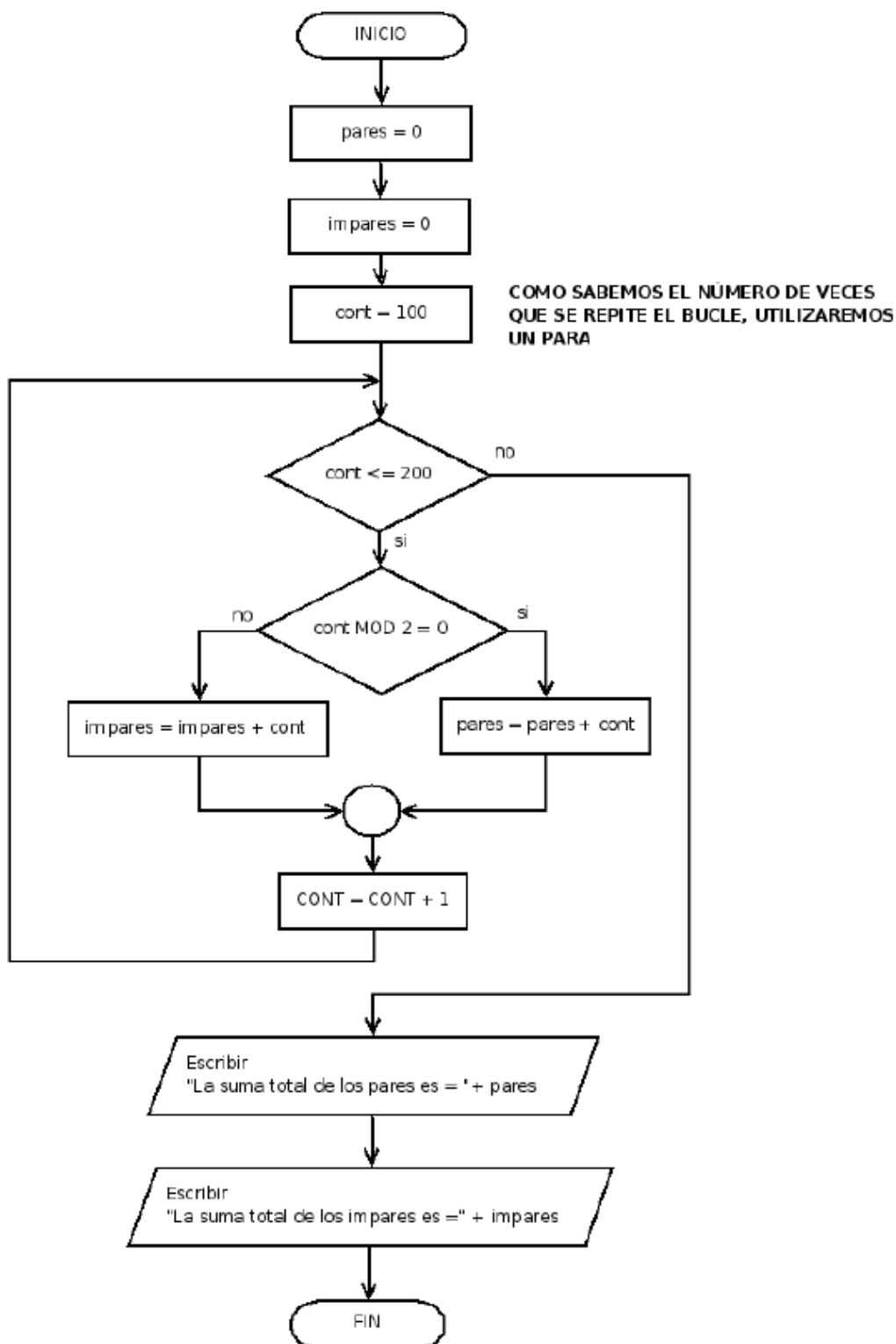


```
12 public class Ejercicio1 {
13
14     public static void main(String[] args) {
15         int cont;
16
17         for(cont=1; cont<=20; cont++)
18             System.out.print(cont + " ");
19
20         System.out.print("\n");
21     }
22 }
```

```
run:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
BUILD SUCCESSFUL (total time: 0 seconds)
```

4.2 Ejemplo 2

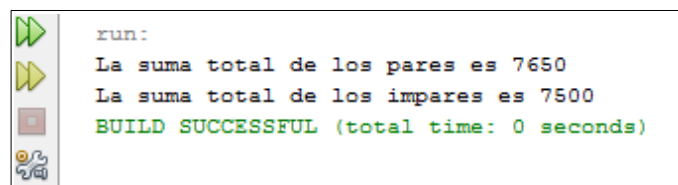
Programa que suma independientemente los pares y los impares de los números comprendidos entre 100 y 200.



Código:

```
12 public class Ejercicio11 {
13
14     public static void main(String[] args) {
15         int pares, impares, cont;
16
17         pares = 0;
18         impares = 0;
19
20         for(cont=100; cont <= 200; cont++)
21         {
22             if(cont % 2 == 0)
23                 pares = pares + cont;
24             else
25                 impares = impares + cont;
26         }
27
28         System.out.println("La suma total de los pares es " + pares);
29         System.out.println("La suma total de los impares es " + impares);
30     }
31
32 }
```

Salida:



```
run:
La suma total de los pares es 7650
La suma total de los impares es 7500
BUILD SUCCESSFUL (total time: 0 seconds)
```

5. OPERACIONES AVANZADAS CON ESTRUCTURAS DE CONTROL

5.1 Aritmética de control en bucles

A. Como la operación de aumentar una variable (**numero = numero + 1**) o disminuir una variable (**numero = numero - 1**) es muy habitual en programación; Java ha incorporado una notación especial para indicar de forma abreviada esa operación. Así:

- *Incrementar* una variable: **numero++**;
- *Decrementar* una variable: **numero--**;

B. También se puede aumentar o disminuir una variable un número de unidades diferente a 1 (**numero = numero +5**), de la siguiente forma abreviada:

- *Incrementar* una variable en 4 unidades: **numero += 4**;
- *Decrementar* una variable en 4 unidades: **numero -=4**;

Hay más operadores similares, consulta la **unidad anterior** para ampliar esta información.

5.2 Contadores y *flags* (variables bandera)

El **contador** es una variable especial, que se utiliza para contar las veces que se repite un suceso dentro de un programa.

Los **contadores**, antes de usarlos deben **inicializarse** (establecer el valor inicial) Esta inicialización puede ser a 0, para poder llevar una cuenta; o, a un valor inicial para otro tipo de cuenta.

Los **contadores**, se **incrementan** o también se **decrementan**: De la forma abreviada como hemos visto en el apartado anterior. Así:

- De un valor al siguiente. Se aumenta la variable contador de 1 en 1.
Ejemplo: **contador++**;
- De un valor a otro valor que se desee.
Ejemplo: **contador+=6**;

Prueba el siguiente código:

```
import java.util.Scanner;

public class Main
{
    public static void main(String[] args)
    {
        int contador = 0;
        while (contador < 5){
            contador ++;
            System.out.println("Contador: " + contador);
        }

        System.out.println("Fin. Contador: " + contador);
    }
}
```

Tras ejecutarlo explica su comportamiento, ¿qué hace?

Las variables **bandera (flags)** son parecidas a los contadores pero se utilizan para mantener el estado de cualquier objeto o situación y dirigir la ejecución del bucle, es decir, marcan la presencia o ausencia de cierta condición. Estas variables DEBEN inicializarse y DEBEN modificarse a lo largo del código para que tengan efectividad.

También pueden utilizarse en otras estructuras, como en las condicionales IF o IF-ELSE, no sólo en los bucles.

En el siguiente ejemplo te mostramos un programa para jugar a adivinar un número del 1 al 10, la variable bandera se llama `"num_usuario"` porque almacena el número que introduce el usuario para tratar de adivinarlo. Esa variable es la que emplea en la condición para que, comparándola con la referencia `"num_adivinar"` el bucle sepa si se cumple y debe continuar de nuevo con el bucle, o si se incumple debe terminar y continuar la ejecución.

Prueba el código que verás a continuación.

Tras eso, trata de modificarlo para que sólo te permita 3 intentos. Si se falla 3 veces el programa debe mostrar un mensaje final y terminar.

```
import java.util.Scanner;

public class Main
{
    public static void main(String[] args)

        int num_adivinar = 7;
        int num_usuario = 0;
        Scanner read = new Scanner(System.in);

        System.out.println("Juego de adivinar el numero del 1 al 10");

        do {
            System.out.println("Introduce un número: ");
            num_usuario = read.nextInt();
            if (num_usuario != num_adivinar)
                System.out.println("Número incorrecto. Otra vez...");
        } while (num_usuario != num_adivinar);

        System.out.println("¡¡Muy bien, has acertado!!");
    }
```

5.3 Bucles infinitos

Los bucles infinitos **son muy peligrosos** y pueden dejar un programa bloqueado en constante ejecución.

Se llega a ellos porque la **condición que mantiene “vivo”** al bucle se cumple de forma constante e ininterrumpida y por lo tanto, como no hay motivos para parar el bucle, éste se ejecuta de forma infinita impidiendo que el programa continúe con el resto de instrucciones.

Esto constituye un fallo muy grave en programación y debe evitarse.

Para evitarlo, siempre hay que tener presente la existencia de una **“condición de terminación”**.

En el siguiente ejemplo, si escribes un número negativo, el programa entrará en bucle infinito.

Una vez lo pruebes, arréglalo para que no suceda eso.

```
import java.util.Scanner;

public class Main
{
    public static void main(String[] args)

        int numero;
        Scanner read = new Scanner(System.in);

        System.out.println("Introduce un número positivo: ");
        numero = read.nextInt();

        while (numero <= 0){
            System.out.println("Este número no es positivo.");
            System.out.println("Por favor, escribe un número positivo.");
        }
}
```

5.4 Condicionales encadenados y condicionales anidados

A veces nos interesa comprobar una cierta cantidad de condiciones relacionadas y elegir una de entre varias acciones. Una manera de hacer esto es **encadenando** una serie de ifs y elses:

```
if (x > 0) {
    System.out.println ("x es positivo");
} else if (x < 0) {
    System.out.println ("x es negativo");
} else {
    System.out.println ("x es cero");
}
```

Además del encadenamiento, es también posible **anidar** un condicional dentro de otro (y otro...). Podríamos haber escrito el ejemplo anterior así:

```
if (x == 0) {
    System.out.println ("x es cero");
} else {
    if (x > 0) {
        System.out.println ("x es positivo");
    } else {
        System.out.println ("x es negativo");
    }
}
```

5.5 Bucles anidados

También podemos encontrar la necesidad de combinar 2 o más bucles entre ellos. Esto ocurre cuando existe la necesidad de que un programa recorra una secuencia de números independiente para 2 o más variables distintas.

Por ejemplo, imagina un programa que quiere mostrar las tablas de multiplicar, desde la del 1 hasta la del 10. Se necesita una variable que lleve la cuenta de qué tabla de multiplicar estamos sacando en cada momento, unida a otra que haga el recorrido del 1 al 10 para cada tabla; la siguiente estructura sería una idea para poder abordarlo:

```
for(tabla=1;tabla<=10;tabla++)
    for(num=1;num<=10;num++)
        System.out.println(tabla + "x" + num + "=" + tabla*num);
```

El primer bucle FOR va a dar 10 vueltas completas, PERO observamos que, cada vuelta que va a dar ese FOR, el segundo también dará 10 vueltas y así sucesivamente, por tanto, ¿cuántas veces se ejecutarán las sentencias que se escriban...? Pues nada menos que $10 \times 10 = 100$ veces. Los bucles anidados, y en general la mayoría de estructuras de control, pueden **componerse**. Es decir, podemos anidar un WHILE y un FOR, o 2 WHILES, o 1 WHILE con un condicional IF y luego otro WHILE....las posibilidades son infinitas.

Veamos otro ejemplo. El siguiente código imprime un cuadrado lleno de asteriscos, cuyo tamaño está determinado por la variable size (asumimos que esta variable ha sido previamente declarada y asignada):

```
for (int i = 1; i <= size; i++)
{
    for (int j = 1; j <= size; j++)
    {
        System.out.print("*");
    }
    System.out.println();
}
```

5.6 Sentencias "break" y "continue"

Java posee 2 sentencias que permiten forzar la salida de un bucle, y estas sentencias son:

- **break** (para salir completamente del bucle)
- **continue** (para salir de la iteración actual y saltar directamente a la siguiente).

Sentencia break

Hace que se salga del bucle inmediatamente por lo que no se ejecutará ni el código que se encuentre después del *break* en esa misma iteración ni ninguna del resto de iteraciones. Es una terminación forzosa del bucle. Útil cuando detectamos alguna circunstancia en la que no queremos que se siga ejecutando. Por ejemplo, este ciclo solo cuenta del 1 al 5, porque la instrucción *break* sale antes de completar el conteo completo:

```
for (int i = 1; i <= 10; i++){  
    System.out.println(i);  
    if (i == 5)  
        break;  
}
```

Sentencia continue

Produce una detención de la iteración actual y salta inmediatamente a la siguiente iteración. Útil cuando detectamos alguna condición que nos permite el salto y no necesitamos que se sigan ejecutando más instrucciones de comprobación. Por ejemplo, este bucle cuenta del 1 al 10, pero no imprime el número 5, porque la instrucción de continuación vuelve al principio del bucle cuando *i* es 5, sin imprimir este valor al final del bucle:

```
for (int i = 1; i <= 10; i++){  
    if (i == 5)  
        continue;  
    System.out.println(i);  
}
```

Estas instrucciones son más frecuentes en bucles, y obviamente dentro de algún *if-else* que controle las circunstancias en que se ejecutarán. No obstante, estas dos instrucciones NO se recomiendan en nuestros programas, ya que se pueden evitar fácilmente para que nuestro código sea más comprensible. En el primer caso, si solo queremos contar del 1 al 5, solo debemos definir un bucle *for* del 1 al 5. En el segundo caso, si no queremos imprimir el número 5 en la pantalla, podemos especificar esta condición en la cláusula *if*:

```
for (int i = 1; i <= 10; i++){  
    if (i != 5)  
        System.out.println(i);  
}
```

5.7 Funcionamiento especial de switch

La estructura **switch** es mucho más potente de lo que parece, hay que probar determinados casos para llegar a dominar todo su potencial. Veamos un ejemplo:

```
switch(nota){  
    case 1:  
    case 2:  
    case 3:  
    case 4: System.out.println("Insuficiente");  
            break;  
    case 5:  
    case 6: System.out.println("Suficiente");  
            break;  
    case 7:  
    case 8: System.out.println("Notable");  
            break;  
    case 9:  
    case 10: System.out.println("Sobresaliente");  
            break;  
    default: System.out.println("La nota es errónea.");  
            break;  
}
```

Casos acumulados hasta un "break". El break es la clave de todo...

6. AGRADECIMIENTOS

Apuntes actualizados y adaptados a partir de la siguiente documentación:

- [1] Apuntes Programación de José Antonio Díaz-Alejo. IES Camp de Morvedre.